soken

Smart contract security review

GBLIN Protocol

Commit `311bd91`, Base (8453)

9 October 2026

0	0	0	2	14
CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL

PREPARED BY SOKEN LTD	CLIENT GBLIN Protocol, gblin.digital
REPOSITORY AND COMMIT github.com/gblinproject/GBLIN-Protocol, 311bd91	NETWORK Base (8453)
SCOPE Seven contracts in production on Base, three libraries and one type	FINDINGS 0 Critical, 0 High, 0 Medium, 2 Low, 14 Informational
DEPLOYED BYTECODE 7 of 7 contracts match the build of `311bd91`	DATE 9 October 2026

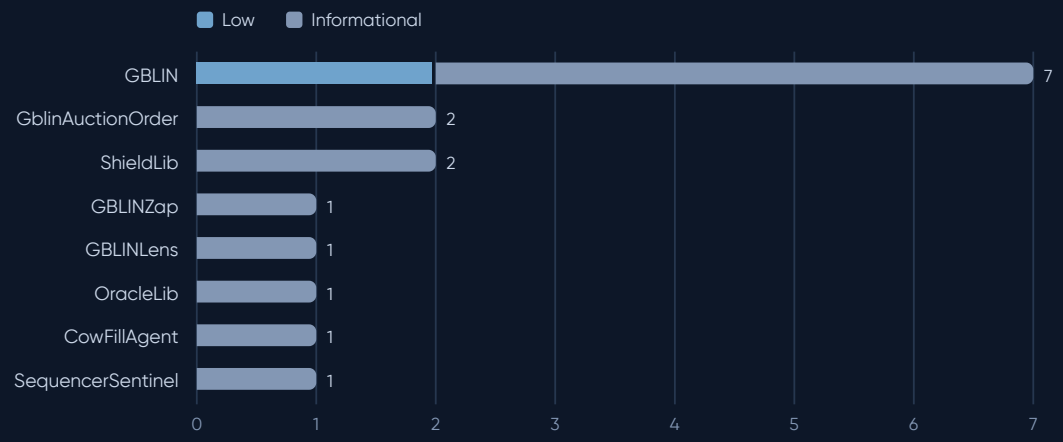
01 Summary

0	0	0	2	14
CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL

7	2,103	112
DEPLOYED CONTRACTS IN SCOPE	LINES OF CODE IN SCOPE	SPECIFICATION STATEMENTS CHECKED

586	84	7 of 7
CLIENT TESTS RUN, 0 FAILED	TESTS WRITTEN BY SOKEN	DEPLOYED BYTECODE MATCHES THE BUILD

Figure 1. Findings by component



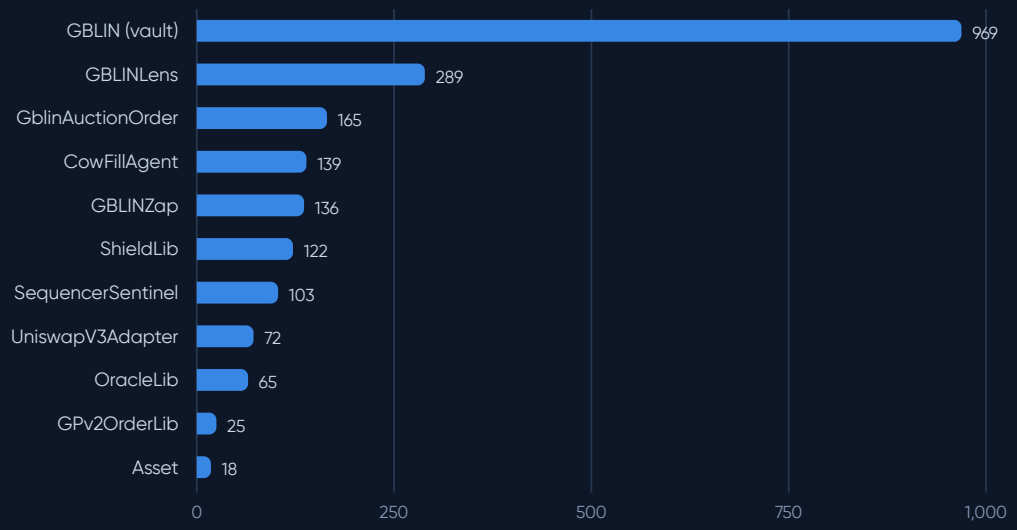
ID	TITLE	COMPONENT	SEVERITY
L-001	Mints accept prices up to 2 hours old; the specification says 30 minutes	GBLIN	LOW
L-002	A delisted row can be abandoned after 26 hours of feed silence; the documentation says seven days	GBLIN	LOW
I-001	`verify()` leaves `kind` and `partiallyFillable` unchecked	GblinAuctionOrder	INFORMATIONAL
I-002	Zap ETH exit sells each leg with `amountOutMinimum = 1`	GBLINZap	INFORMATIONAL
I-003	Launch configuration differs from the code defaults; the launch `setParam` calls are not listed	GBLIN	INFORMATIONAL
I-004	Lens views can answer where the vault would revert	GBLINLens	INFORMATIONAL
I-005	Crash shield events are not declared in `IGBLIN`	ShieldLib	INFORMATIONAL
I-006	`OracleLib` feed reads are not gas-capped	OracleLib	INFORMATIONAL
I-007	Views return intermediate values during the bid callback	GBLIN	INFORMATIONAL

I-008	Order generator duplicates vault constants and reads decimals from the token	GblinAuctionOrder	INFORMATIONAL
I-009	Relayer allowance stays until the fill is closed	CowFillAgent	INFORMATIONAL
I-010	README omits the lower bound of the shield base threshold	GBLIN	INFORMATIONAL
I-011	SequencerSentinel implements a subset of the aggregator interface	SequencerSentinel	INFORMATIONAL
I-012	Two small gas items	GBLIN	INFORMATIONAL
I-013	README gives the shield threshold range as 15% to 50%; the code also caps it at the full-slash drawdown	ShieldLib	INFORMATIONAL
I-014	README says the shield refreshes on every redemption; `sellGBLIN` does not refresh it	GBLIN	INFORMATIONAL

02 Scope

CONTRACT	FILE	SLOC	ADDRESS
GBLIN (vault)	`src/GBLIN.sol`	969	`0xc2181d...6B1D53`
GBLINLens	`src/periphery/GBLINLens.sol`	289	`0xfCFea8...D26f61`
GblinAuctionOrder	`src/fillers/GblinAuctionOrder.sol`	165	`0x156Ffd...1ddaB9`
CowFillAgent	`src/fillers/CowFillAgent.sol`	139	`0x0f4307...7E2fc8`
GBLINZap	`src/periphery/GBLINZap.sol`	136	`0x0E9D6C...e60200`
SequencerSentinel	`src/SequencerSentinel.sol`	103	`0x9F13C5...0D3F67`
UniswapV3Adapter	`src/adapters/UniswapV3Adapter.sol`	72	`0x062654...Cd9d3F`
ShieldLib	`src/libraries/ShieldLib.sol`	122	Library, part of the vault
OracleLib	`src/libraries/OracleLib.sol`	65	Library
GPv2OrderLib	`src/libraries/GPv2OrderLib.sol`	25	Library
Asset	`src/types/Asset.sol`	18	Type
Total		**2,103**	

Figure 2. Size of the code in scope, source lines without comments and blank lines

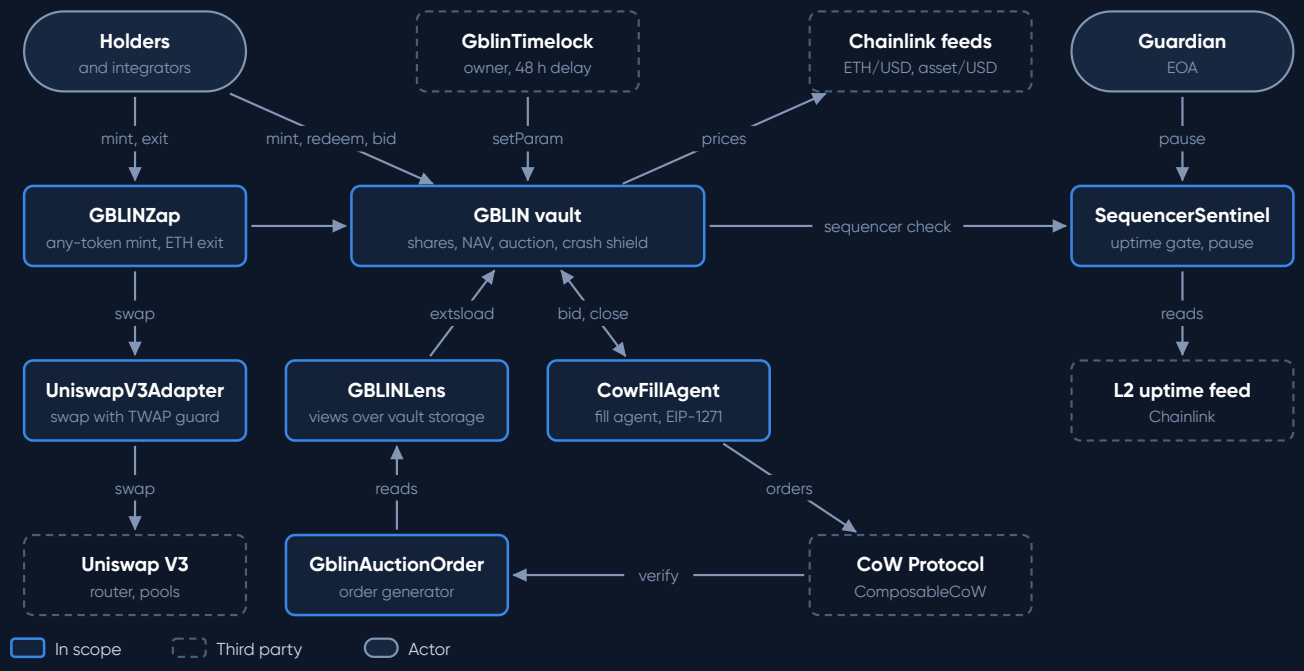


Out of scope

ITEM	REASON
`AerodromeAdapter`	Not listed in `docs/deployments.md` and not used by the deployed Zap, whose adapter is the UniswapV3Adapter; the client states it is not deployed
`legacy/` directory	Previous vault versions
`GblinTimelockController`, the vault owner	Listed in the Timelock section of `docs/deployments.md`; its source is not in the repository at `311bd91`. A 182 line extension of OpenZeppelin TimelockController v5.6, source verified on Sourcify. The extension was read to describe governance in section 03; it was not tested and its OpenZeppelin base was not reviewed
`lib/` (OpenZeppelin, Solady)	Third-party code. Reviewed only for how the contracts use it
CoW Protocol settlement, `ComposableCoW`, Uniswap V3, Chainlink feeds, WETH	Third-party contracts
CoW watch-tower, other off-chain components, web front end	Off-chain

03 Architecture and privileged roles

Figure 3. Contracts in scope and their external dependencies



Privileged roles

ROLE	HOLDER	TYPE	POWER
Vault owner	0x6aBeC8716fFeEcf7C3D6e68255b4797113E8e5Dd	Timelock: OpenZeppelin TimelockController v5.6 with a constant 48 h delay and a 14 day grace period, self-administered	Every owner action, each after 48 h: parameters within bounds, base weights, asset listing, delisting, relisting and abandoning, oracle replacement, the addresses of the fill agent, the ETH feed, the sequencer feed and the fee recipient, transfer of ownership, and registration of the CoW orders on the fill agent
Proposer	0xc7AeFcfC871bA51FA2ddfE3117b69AC055C98eF1	Safe 1.5.0, 2 of 3 signatures	Queues operations in the timelock
Canceller	0xc7AeFcfC871bA51FA2ddfE3117b69AC055C98eF1	The same Safe	Cancels queued operations
Executor	Open	Any address	Executes an operation after its 48 h delay and before it expires 14 days later
Sentinel owner	0x6aBeC8716fFeEcf7C3D6e68255b4797113E8e5Dd	The same timelock	Pauses without a budget, each call capped at 30 days; lifts any pause; appoints the guardian
Sentinel guardian	0x30590c0D05c26562d7296CE3D927d3418d2e6dcA	EOA, also one of the three Safe owners	Reports the sequencer as down, which stops mints and auction fills and leaves redemptions open. One stretch of at most 30 days, then 7 days of rest. Lifts only a pause it opened

OBSERVATION	DETAIL
Exit is always open	Redemption in kind cannot be paused. Every owner action waits 48 h, so holders can leave before it takes effect.
No upgrade path	No proxy. Code cannot be changed in place.
Proposer and canceller	One Safe holds both roles and needs two of three signatures. Owners: 0x9FFa542E369C53af62380296092EC669f329a9ee (also the fee recipient), 0x30590c0D05c26562d7296CE3D927d3418d2e6dcA (also the sentinel guardian) and 0xb36686A284e086091Fa013645B47Cf156f4365EE. The executor role is open; the admin role is held by the timelock. Read on chain on 9 October 2026.
Separation of the roles	The timelock checks that a canceller is not a proposer only in its constructor; a later grant is not checked. With one Safe in both roles, an operation queued by the Safe can be cancelled only by the same Safe.

04 Method and severity scale

STEP	WORK DONE
Area review	Line-by-line reading of every in-scope file in four areas: vault accounting; auction, shield and oracle; CoW path; periphery. The interfaces under `src/interfaces/` were read as the API the contracts implement
Cross-area review	Attacker goals across contracts; specification against code (112 statements from the README, the docs and the NatSpec); assembly and low-level calls
Testing	Tests written for the statements that can be tested: 70 unit tests, 7 fuzz tests, 7 invariant campaigns. Client suite run locally and on a Base fork
Static analysis	Slither and Aderyn, used as leads only. No tool output is reported as a finding
Deployment verification	Build of `311bd91` compared with the deployed bytecode; configuration and roles read on chain
Known issues	The six items of `KNOWN_ISSUES.md` are treated as known and are not reported again

Severity scale

SEVERITY	MEANING
CRITICAL	Loss or permanent freeze of holder funds with no unusual precondition
HIGH	Loss or freeze under realistic preconditions
MEDIUM	Limited loss, or loss that needs an uncommon but plausible condition inside the protocol's control
LOW	Limited loss that needs a failure outside the protocol, or a departure from the specification with a bounded effect
INFORMATIONAL	Documentation, hardening and integration notes

05 Findings

Low

L-001 Mints accept prices up to 2 hours old; the specification says 30 minutes

LOW

COMPONENT	LOCATION	CATEGORY
GBLIN	`src/GBLIN.sol:448`, `:454`, `:486`, `:1139–1156`, `:1179–1184`; `README.md:69`	Oracle freshness

DESCRIPTION

README: "Mints and bids move value at a price and require fresher feeds: 30 minutes at launch for the feeds of non-stable assets." In the code only `bid` applies that window.

PATH	CHECK	LIMIT	LIVE VALUE
`bid`	`_tradeValue`	`maxOracleAgeTrade`	1,800 s
`buyGBLINWithWeth`, `_mintGBLIN`	`_requireFreshOracles`	`oracleTimeout`	7,200 s
`buyGBLINInKind`	`_requireFreshOracles` and line 454	`oracleTimeout`	7,200 s

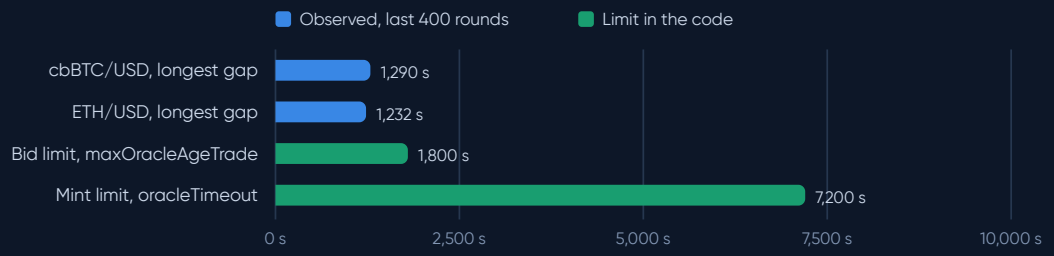
A mint is priced on a feed that a bid in the same block rejects.

IMPACT

A feed between 30 minutes and 2 hours old that differs from the market by `m` misprices NAV, and the minter receives the difference from the existing holders. Figures for `m` = 5%:

DIRECTION	GAIN OF THE MINTER	BASIS
Mint with WETH, feed of a non-stable row below the market	2.25% of the deposit before the 0.10% mint fee	cbBTC target weight 45%: 0.45 * 5%
Mint with WETH, ETH/USD feed above the market	2.75% of the deposit before fees	Rows other than WETH are 55% of the target weights: 0.55 * 5%
In-kind deposit, feed of the deposited asset above the market	`m * (1 - w)` less the in-kind fee	`w` is the weight of the asset
Test, two rows of about half of NAV each, feed 45 minutes old	2.49% on the WETH path, 2.10% on the in-kind path, fees included	`VerifyL001Directions.t.sol`

Figure 4. Feed age limits in the code against the longest update gap observed on Base



Measured on 3 October 2026 over the last 400 rounds of each feed; the samples span 120 hours (cbBTC/USD) and 65 hours (ETH/USD). No gap exceeded 1,800 s, so the window opens only when a feed is delayed or halted. That dependency on a failure outside the protocol sets the severity at Low.

RECOMMENDATION

OPTION	CHANGE	EFFECT
Next deployment	Price the mint paths with the trading age	The code follows the README
Documentation	State in the README that the 30 minute window applies to bids only	The README follows the code
Deployed vault	Set both ages of key 6 to 30 minutes through the timelock	Mints follow the README at once. `isNavReliable()` then turns false after 30 minutes of feed age instead of 2 hours, which stops the Lens quotes and the NAV floor of the Zap in that state. Margin over the longest observed gap: about 8 minutes

EVIDENCE

`A10racleFreshness.t.sol`, 2 of 2 passing: with the cbBTC feed 45 minutes old and the ETH feed fresh, a mint through `buyGBLIN` succeeds and `bid` reverts with `PriceUnavailable`.

L-002 A delisted row can be abandoned after 26 hours of feed silence; the documentation says seven days

LOW

COMPONENT	LOCATION	CATEGORY
GBLIN	`src/GBLIN.sol:930`; `src/libraries/ShieldLib.sol:48`, `:140–148`; `README.md:158`; `docs/governance.md:53`	Governance safeguard

DESCRIPTION

README: "abandoning requires a delisted row whose feed has given no price for seven days". The code does not measure the silence of the feed.

CONDITION IN `ASSETACTION(3, I)`	WHAT IT MEASURES
`_price(a.oracle) == 0`	The feed has no usable price at the moment of the call: older than 26 hours (`PRICE_MAX_AGE`), at a bound, non-positive or reverting
`block.timestamp - a.lastPeakUpdate >= 7 days`	Time since the shield last moved the peaks of the row, which happens on a new high or after a full day. The shield skips rows with zero base weight, and delisting sets the base weight to zero, so the value stops moving once the row is delisted. `updateOracle` resets it

For a row delisted seven or more days ago the second condition is true whatever the feed did in that time, unless the oracle was replaced. The abandon then needs only 26 hours of feed silence.

IMPACT

An abandoned row is excluded from NAV (`:813`) and from redemption (`:594`) for good, and no function recovers it, so its remaining balance is lost to the holders. The documented safeguard of seven days is 26 hours for a row that has been sold down through the auction for more than a week. The call is an owner action behind the 48 h timelock. With the open executor role anyone can execute a queued abandon once the feed has been silent for 26 hours, within the 14 days before the queued operation expires. The loss is bounded by the balance left in the delisted row.

RECOMMENDATION

In a next deployment, read the feed's own `updatedAt` from `latestRoundData` and require it to be at least seven days old, so that a feed pinned at a bound also counts its real age; decide explicitly how a feed that reverts is treated. On the deployed vault, state the actual rule in the README and in `governance.md`.

EVIDENCE

`VerifyAbandonSilence.t.sol`, 2 of 2 passing: a row whose feed answered every day for eight days after the delisting and then stopped for 27 hours is abandoned, nine days and three hours after the delisting; the same 27 hours right after the delisting are refused.

Informational

I-001 `verify()` leaves `kind` and `partiallyFillable` unchecked

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GblinAuctionOrder	`src/fillers/GblinAuctionOrder.sol:160–195`	Input validation

DESCRIPTION

`verify()` does not check the order fields `kind` and `partiallyFillable`. The NatSpec states that this is intentional, since an order is accepted only in the block its fill was opened.

IMPACT

No loss path found: quantity is capped by the relayer allowance and price by the ratio check. The two fields change how the settlement reads the amounts, so the handler depends on settlement internals.

RECOMMENDATION

Require `KIND_SELL` and `partiallyFillable == true`.

I-002 Zap ETH exit sells each leg with `amountOutMinimum = 1`

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GBLINZap	`src/periphery/GBLINZap.sol:188`; `src/adapters/UniswapV3Adapter.sol:75–80`; `README.md:59`; `KNOWN_ISSUES.md` items 1 and 5	Slippage

DESCRIPTION

`sellGBLINForEth` checks the caller's floor once, on the total. The bound per leg is the adapter's TWAP guard, which compares the pool's current tick with its own time-weighted average before the swap; it does not read the oracle and does not limit the price impact of the swap itself. Item 1 of `KNOWN_ISSUES.md` names the caller's minimum and the TWAP band, and the README (line 103) the NAV floor. The README (line 59) says the TWAP band refuses "a pool price too far from the oracle", which the adapter does not do, and item 5 of `KNOWN_ISSUES.md` says "no leg is ever sold without a bound", which reads as a price bound per leg.

IMPACT

In a thin pool or a wrong fee tier, a leg that is a fraction `w` of the redemption can be sold up to about `maxSlippageBps / w` below the feed price. The loss is the caller's. While `isNavReliable()` is true it does not exceed 100 bps of the whole exit, the documented limit of the Zap. While it is false, the only floor is the caller's `minEthOut`. The figures below come from a test adapter that sells a leg at a set discount; the UniswapV3Adapter and its TWAP guard are not part of that test.

LEG	WEIGHT	RESULT AT THE LIVE 100 BPS
USDC	10%	Sold 9% below the feed, exit succeeded 90 bps below NAV. At 11% the exit reverted
cbBTC	45%	Limit between 2% and 2.5% below the feed

RECOMMENDATION

Set a per-leg minimum from the oracle price, or reword item 5 of `KNOWN_ISSUES.md` and line 59 of the README to describe the TWAP band as a check of the pool against its own average.

EVIDENCE

`VerifyZapLegBound.t.sol`, 3 of 3 passing, with the test adapter described above.

I-003 Launch configuration differs from the code defaults; the launch `setParam` calls are not listed

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GBLIN	`src/GBLIN.sol:137`, `:161`; `docs/deployments.md`	Configuration

DESCRIPTION

PARAMETER	CODE DEFAULT	LIVE VALUE
`auctionCapBps`	50 bps	25 bps
`assetListingDelay`	172,800 s (48 hours)	86,400 s (24 hours)

Storage started at the code defaults: at the creation block 51563253 the vault held 50 bps and 172,800 s. The deployer sent 33 `setParam` transactions between blocks 51563358 and 51565793. The final values match "Parameters at launch" in `docs/deployments.md`. All of them precede the timelock taking ownership at block 51660727. No `ParamUpdated` event exists between block 51565793 and block 52110575, so no parameter has changed under the timelock.

IMPACT

None on the vault in service. A redeployment from this source without the same calls starts with the code defaults.

RECOMMENDATION

Set the defaults to the launch values, or list the launch transactions in `deployments.md`.

I-004 Lens views can answer where the vault would revert

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GBLINLens	`src/periphery/GBLINLens.sol:133–143`, `:287–313`, `:383–391`	View consistency

DESCRIPTION

`quoteBuy` checks `isNavReliable()`, which does not include the sequencer check that the mint paths apply. `auction()` values rows through `_ethValue`, which requires only a positive answer, with no staleness or bound check.

IMPACT

An integrator can receive a quote or a gap for a call the vault then rejects. No on-chain loss. The order generator has its own checks.

RECOMMENDATION

Have `quoteBuy` revert while the vault's sequencer check fails, and value rows in `auction()` with the vault's age and bound checks; or state both differences in the Lens NatSpec.

I-005 Crash shield events are not declared in `IGBLIN`

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
ShieldLib	`src/libraries/ShieldLib.sol:33–35`	Events

DESCRIPTION

`CrashShieldActivated` and `CrashShieldDeactivated` are emitted in the vault's context but are absent from the interface.

IMPACT

A consumer that builds event filters from `IGBLIN` misses shield transitions.

RECOMMENDATION

Declare both events in `IGBLIN`.

I-006 `OracleLib` feed reads are not gas-capped

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
OracleLib	`src/libraries/OracleLib.sol:27–33, :42, :55, :91`	Liveness

DESCRIPTION

`withinBounds` and `hasAggregator` use plain staticcalls, and the `latestRoundData` reads in `price()` and `age()` carry no gas cap either, unlike the gas-capped calls the vault uses for tokens and the fill agent.

IMPACT

Liveness only, and only with a misbehaving feed contract.

RECOMMENDATION

No change is required for Chainlink feeds. If a gas cap is added, a read that runs out of gas must count as a failed check: `withinBounds` returns true on a failed call, so a plain cap would skip the bounds check.

I-007 Views return intermediate values during the bid callback

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GBLIN	`src/GBLIN.sol:663–665`	Read-only reentrancy

DESCRIPTION

The mint, redemption, bid, claim and `refreshWeights` entry points are guarded by `nonReentrant`. The views `totalEthValue` and `navPerShare` are not. Inside `onAuctionFill` the vault has sent its output and has not yet pulled the input, so both views read low by the value of the output. The ETH send of `sellGBLIN` does not have this effect: the shares are burned and every token leg is delivered before the send.

IMPACT

A third-party protocol that reads NAV inside the callback gets a wrong value.

RECOMMENDATION

State it in the NatSpec of both views. The Lens already exposes `reentrancyLocked`.

EVIDENCE

`VerifyBidCallbackWindow.t.sol`: inside `onAuctionFill` `totalEthValue` reads lower than before the bid by exactly the output amount, and both views are restored after the call. `VerifyEthSendWindow.t.sol`: a receiver that reads the views from `receive()` gets the final values.

I-008 Order generator duplicates vault constants and reads decimals from the token

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GblinAuctionOrder	`src/fillers/GblinAuctionOrder.sol:43–45, :111`	Code duplication

DESCRIPTION

The vault has one 26 hour window, the constant `PRICE_MAX_AGE`, which it also uses for the feed of a stable asset. The generator carries its own copies of it (`STABLE_FEED_MAX_AGE` and `PRICE_MAX_AGE`, both 26 hours) and reads token decimals from the token, while the vault uses the decimals it stored at listing.

RECOMMENDATION

Expose the stored decimals through the Lens and read them there. The constant is private and cannot be read from outside, so mark the copies with a cross-reference in both files.

I-009 Relayer allowance stays until the fill is closed

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
CowFillAgent	`src/fillers/CowFillAgent.sol:88`	Allowance

DESCRIPTION

If the post-hook does not run, the allowance for the fill amount remains until the vault closes the fill or `emergencyClose` is called.

IMPACT

No exploitable window found: no order validates outside the opening block.

RECOMMENDATION

No change required.

I-010 README omits the lower bound of the shield base threshold

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GBLIN	README section 10, key 2; `src/GBLIN.sol:966`	Documentation

DESCRIPTION

The README gives no lower bound for the shield base threshold. The code enforces one, and `governance.md` and the NatSpec state it correctly.

RECOMMENDATION

Add the lower bound to the README.

I-011 SequencerSentinel implements a subset of the aggregator interface

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
SequencerSentinel	`src/SequencerSentinel.sol:15`	Interface

DESCRIPTION

`version()` and `getRoundData()` are missing. The vault does not use them; off-chain tools might.

RECOMMENDATION

Implement both, or note the subset in the NatSpec.

I-012 Two small gas items

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GBLIN	`src/GBLIN.sol:527–529`; `lib/solady/src/tokens/ERC20.sol:355`	Gas

DESCRIPTION

ITEM	DETAIL
`_constantNameHash`	Not overridden, so the name hash is recomputed on each domain separator computation
`_accrueManagementFee`	Writes `lastManagementFeeAccrual` before its early return, so a second call in the same block stores the same value again

RECOMMENDATION

Override `_constantNameHash`. In `_accrueManagementFee`, return before the write when the stored time equals `block.timestamp`; the write on the first call must stay, since it starts the clock.

I-013 README gives the shield threshold range as 15% to 50%; the code also caps it at the full-slash drawdown

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
ShieldLib	`src/libraries/ShieldLib.sol:75`; `README.md:123`	Documentation

DESCRIPTION

README: the threshold is "bounded between 15% and 50%". After the clamp to `maxCrashBps` the code clamps the threshold to `fullSlashDrawdownBps`. Live values read through the Lens: minimum 1,500 bps, maximum 5,000 bps, full-slash drawdown 3,000 bps. The effective range is 15% to 30%.

IMPACT

None on funds. The shield activates once the drawdown exceeds 30% at the latest, earlier than the README states.

RECOMMENDATION

State in the README that the threshold never exceeds the full-slash drawdown.

I-014 README says the shield refreshes on every redemption; `sellGBLIN` does not refresh it

INFORMATIONAL

COMPONENT	LOCATION	CATEGORY
GBLIN	`src/GBLIN.sol:553–565`; `README.md:125`; `src/interfaces/IGBLIN.sol:338–339`	Documentation

DESCRIPTION

README: "The shield refreshes on every mint, redemption, bid and on `refreshWeights()`". `sellGBLIN` closes an open fill and wraps ETH (`_settleAndWrap`) and does not call `_refreshShield`. The NatSpec of `IGBLIN` states it correctly: the shield and the auction state are updated by the next call that prices the basket.

IMPACT

None. Redemption is pro rata in kind and does not use the weights.

RECOMMENDATION

Remove "redemption" from that sentence of the README.

06 Deployment verification

CONTRACT	LOCAL BUILD, BYTES	ON CHAIN, BYTES	IMMUTABLE REFERENCES	DIFFERING BYTES
GBLIN	22,392	22,392	19	0
GBLINLens	6,559	6,559	0	0
GBLINZap	6,573	6,573	33	0
SequencerSentinel	2,285	2,285	3	0
UniswapV3Adapter	2,802	2,802	8	0
CowFillAgent	5,595	5,595	20	0
GblinAuctionOrder	8,191	8,191	24	0

CHECK	RESULT
Runtime bytecode	All seven deployed contracts equal the local build of `311bd91`, byte for byte, with the immutable slots masked
Vault creation transaction	Input of `0x5419f2da2a9eb58539453efe76515791ce549c4ce31a1cd0c696200c0d78a666` equals the local creation bytecode followed by 128 bytes of constructor arguments
Vendored dependencies	`ReentrancyGuard`, `SafeERC20`, `Math` and `Panic` are identical in the vendored copy and in OpenZeppelin v5.7.0. `StorageSlot` and `SafeCast`, which they import, differ in one comment line each
Ownership	Vault owner is the timelock (`GblinTimelockController`), `minDelay` 172,800 s, grace period 1,209,600 s; no pending owner
Upgradeability	No proxy and no upgrade path
Wiring	Vault, Lens, Zap, adapter, sentinel, fill agent and order generator point at each other and at the canonical WETH, Uniswap V3, Chainlink and CoW Protocol addresses on Base

07 Testing

RUN	SUITES	TESTS	PASSED	FAILED
Client suite, quick pass without a fork (`ForkBase` and `AuctionOnFork` excluded by path)	86	547	547	0
Client suite, complete, Base fork at block 51,340,251, gas limit raised	91	586	586	0
Soken tests: 70 unit, 7 fuzz, 7 invariant	15	84	84	0

Figure 5. Coverage of the client suite by file, run without a fork, 545 tests



Measured with `--ir-minimum`, which attributes hits loosely, so the line figures are a lower bound. Two examples of lines reported as not covered that the suite does execute: the "fill open" revert of `GblinAuctionOrder` (asserted by `OrderGenerator.t.sol`) and the balance read in assembly of the Zap (runs on every exit to ETH). The only functions never called are `decimals()` and `description()` of the sentinel.

FILE	LINES	BRANCHES	FUNCTIONS
`GBLIN.sol`	87.8% (556/633)	87.5% (168/192)	100% (78/78)
`GBLINLens.sol`	95.2% (119/125)	100% (10/10)	100% (31/31)
`GBLINZap.sol`	91.8% (78/85)	77.8% (21/27)	100% (5/5)
`SequencerSentinel.sol`	88.7% (55/62)	95.7% (22/23)	81.8% (9/11)
`UniswapV3Adapter.sol`	100% (30/30)	100% (6/6)	100% (4/4)
`CowFillAgent.sol`	82.7% (67/81)	100% (14/14)	100% (14/14)
`GblinAuctionOrder.sol`	97.9% (92/94)	83.3% (25/30)	100% (7/7)
`ShieldLib.sol`	100% (80/80)	96.0% (24/25)	100% (4/4)
`OracleLib.sol`	90.2% (37/41)	68.4% (13/19)	100% (6/6)
`GPv2OrderLib.sol`	100% (2/2)	none	100% (1/1)

Observations on the test suite

TOPIC	OBSERVATION
Invariant handler	Takes feeds down by zeroing the price. A feed that is stale by age with a valid price is not exercised, the sequencer is not flipped and the one hour grace period is not crossed. L-001 sits in that gap
Mock token	`MockToken` applies its blacklist in `transfer` but not in `transferFrom`. USDC applies it in both
Fork files	Ten files need a fork: the six named in the README of the test suite (`ForkBase`, `AuctionOnFork`, `NightAttack`, `NightAttack2`, `PenTestEconomic`, `RealScaleSimulation`) and `NightAttack3`, `FullRehearsal`, `OracleIdentity`, `RealHeartbeat`. They hold 87 tests; all pass at block 51,340,251. The internal suite of the client holds 591 tests; the five of `MigrationFromLegacy.t.sol` test the legacy contracts, which are out of scope, and are not in the 586.
Early returns	48 of the 547 tests of the quick pass are fork tests that return early without `--fork-url`. They are counted as passed and assert nothing in that run
Gas limit	`test_A34_matrixCap25Ramp6hStab5` and `test_A37_feedBelowTheMarketThenUpdated` use 1,577,118,820 and 1,328,118,760 gas and fail under Foundry's default limit of 1,073,741,824. The fork run in the table above was made with the limit raised
Reproducibility	Without a pinned block the `AuctionOnFork` results change from run to run. A fork block and a gas limit in `foundry.toml` make the fork run reproducible
Coverage tooling	`forge coverage` fails with "stack too deep" at `src/GBLIN.sol:648` without `--ir-minimum` and in `GBLINZap.sellGBLINForEth` with it. Coverage was measured on a copy in which the body of the leg loop of `sellGBLINForEth` is an internal function, logic unchanged, with the two bytecode-size tests left out

08 Review coverage

AREA	CHECKED	RESULT
Share and NAV accounting	Rounding in every mint and redeem path; virtual offset (1e6 shares, 1 wei) against first-depositor inflation; fee shares; management fee accrual across rate changes; credits and reserved balances	I-012
Redemption in kind	Not gated by feeds, fees, the sentinel or an open fill after its opening block; gas-capped transfers and the credit fallback; hostile tokens (no return data, short and oversized return data, reverting, gas-burning)	No issue found
Dutch auction	Gap and affordability bounds; premium curve including the cycle wrap; callback path; mint, bid and redeem sequences in one transaction	I-007
Fill agent path	Mint, bid and redemption close an open fill (`_settleFill`) before they price anything, and `CowFillAgent.close()` reverts in the opening block while the swap is incomplete. Gas starvation of `close()` tested from 30,000 to 3,000,000 gas in steps of 5,000 on a mint, a redemption and `refreshWeights`: every call that completed had first returned the fill's tokens to the vault	No issue found
CoW integration	`GPv2OrderLib` type hash and struct hash against the canonical definitions; the EIP-1271 chain; replay across buckets and after close; token custody of the agent; the deployed wiring read on chain	I-001, I-008, I-009

Oracle library	`price()` and `age()` do not revert on a broken feed; the bounds check requires an answer strictly inside the aggregator limits; conversion precision and overflow for 8 and 6 decimal assets; the two age tiers	L-001, I-006
Crash shield	Peak decay across a feed outage; activation and recovery; redistribution to healthy stable rows only (item 4 of `KNOWN_ISSUES.md` is closed in this code)	I-005, I-013
EIP-3009	Type hashes; validity bounds; nonce handling; malleability; the ERC-1271 path; domain separator after a chain id change (`VerifyDomainSeparatorChainId.t.sol`, 2 of 2 passing); Permit2 infinite allowance disabled	No issue found
Lens	Every storage slot it reads against the compiler's storage layout, including the packing of the fill slot and of the 12-slot basket row; on Base at block 52,195,020, every scalar slot and every basket row slot it reads, raw and through the deployed Lens, with equal results; the mapping slots for the three basket tokens	I-004
Zap	Fee-on-transfer tokens rejected; approvals to the adapter reset after each swap; unused input returned; receiver checks; all-or-nothing exit to ETH; ETH accounting excludes a prior balance; reentrancy guard on both entry points	I-002
Swap adapter	TWAP tick averaging and its rounding for negative deltas; deviation check; approval to the router reset after the swap; zero output rejected	No issue found
Sentinel	The guardian budget and the owner cap cannot be exceeded by sequencing calls; pass-through of the real feed when not paused; redemptions are not gated	I-011
Specification	112 statements from the README, the docs and the NatSpec against the code and the chain	L-001, L-002, I-003, I-005, I-010, I-013, I-014

09 Disclaimer

This report was prepared by SOKEN LTD (Companies House 15253906). It records a security review of the source code at commit 311bd91 and of the contracts deployed on Base (8453), dated 9 October 2026. A security review is a point-in-time assessment of the code in scope and is not a guarantee that the code is free of defects, nor an endorsement of the project, its economics or its operators. Statements about on-chain configuration describe the state on the date of reading; on-chain values change, so re-read any figure at the linked address before relying on it. Off-chain key management, operational practice, the front end and third-party protocol risk are outside the scope of this review. Nothing in this document is financial, investment or legal advice.

SOKEN LTD | 86-90 Paul Street, London EC2A 4NE | Companies House 15253906
soken.dev | x.com/soken_team | t.me/soken_ai | linkedin.com/company/soken-llc | github.com/Soken-LTD